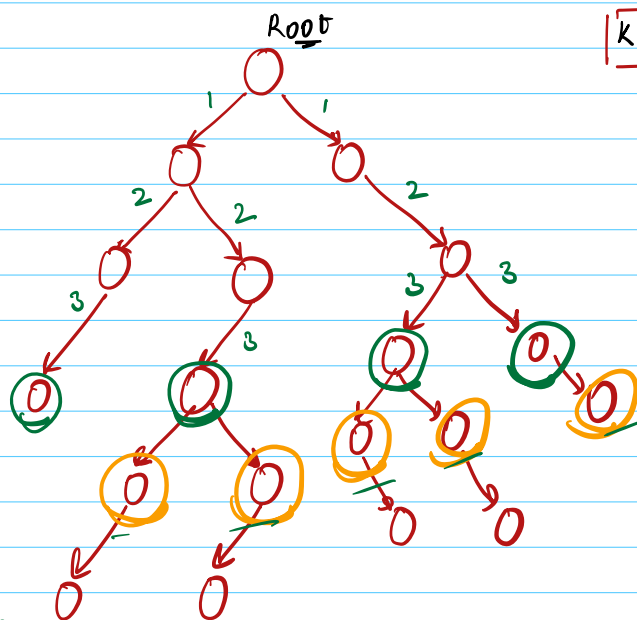


Agenda:

- 1) count no. of nodes at distance K from the root node.
- 2) count no. of nodes at distance K from any node.
- 3) Longest path across the root.
- 4) Longest path across any node
- 5) Max path sum (H.W)

Q 17



$K=3 \rightarrow 4$

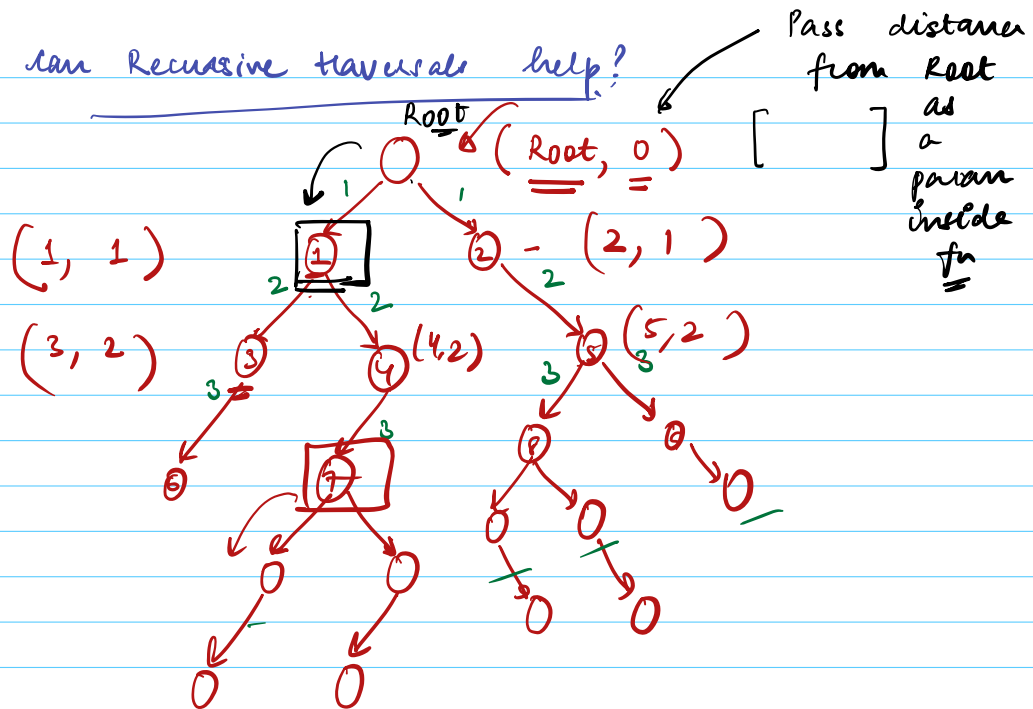
$K=4 \rightarrow 5$

1st Approach

$\Rightarrow$ Level order Traversal ✓

✓

Can Recursive traversal help?



Global list = []

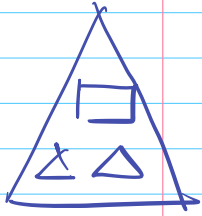
void count (Root, dist)

if (root == NULL) return;

if (dist == K) list.add(root) return.

count (root.left, dist + 1)

count (root.right, dist + 1)



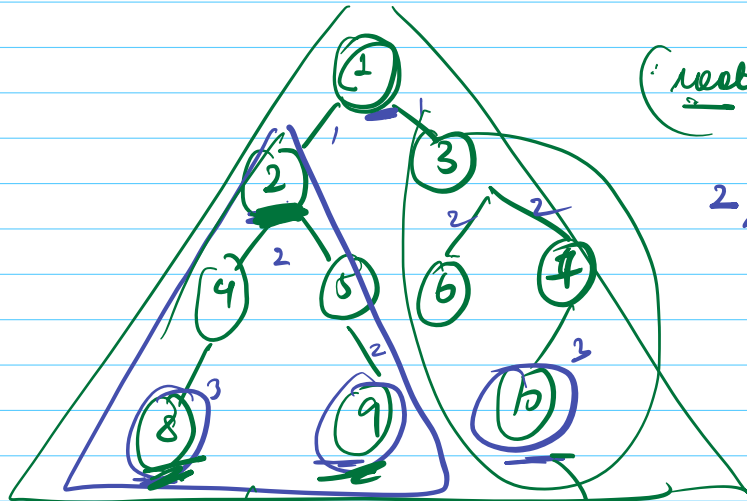
Assumption: returns count of nodes at distance k from root.

```

int count (root k) {
    if (root == NULL) return 0;
    if (k == 0) return 1;
    //
    x = count (root->left, k-1)
    y = count (root->right, k-1)
    return x + y;
}

```

1



(root, 3)

2, 1

nodes at distance
2 from Node 2



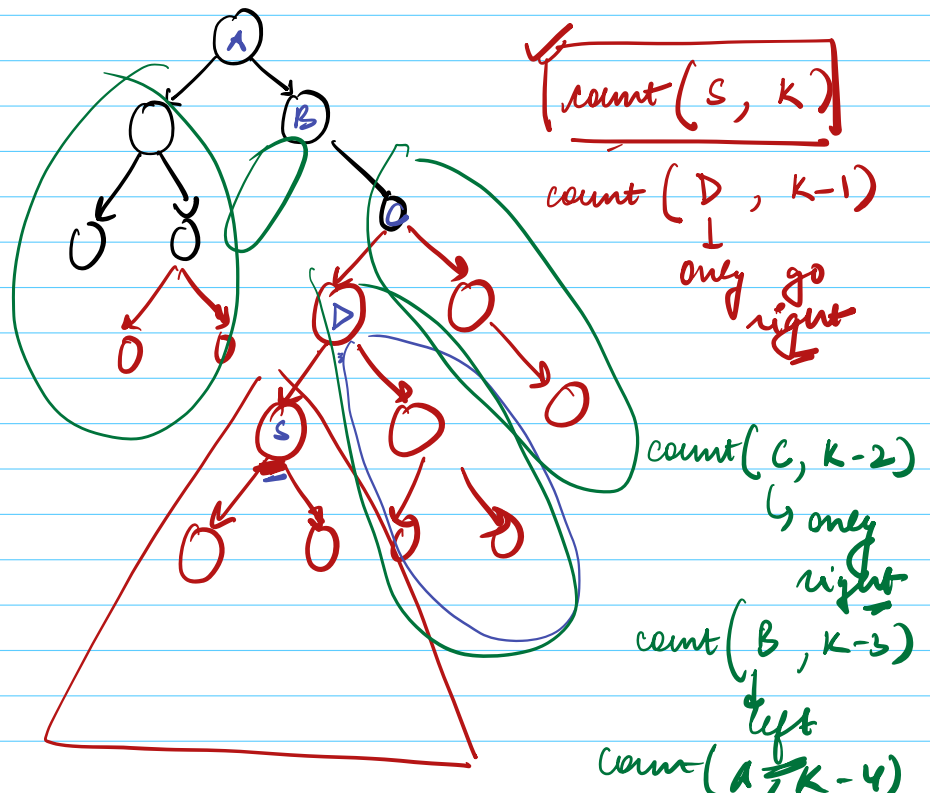
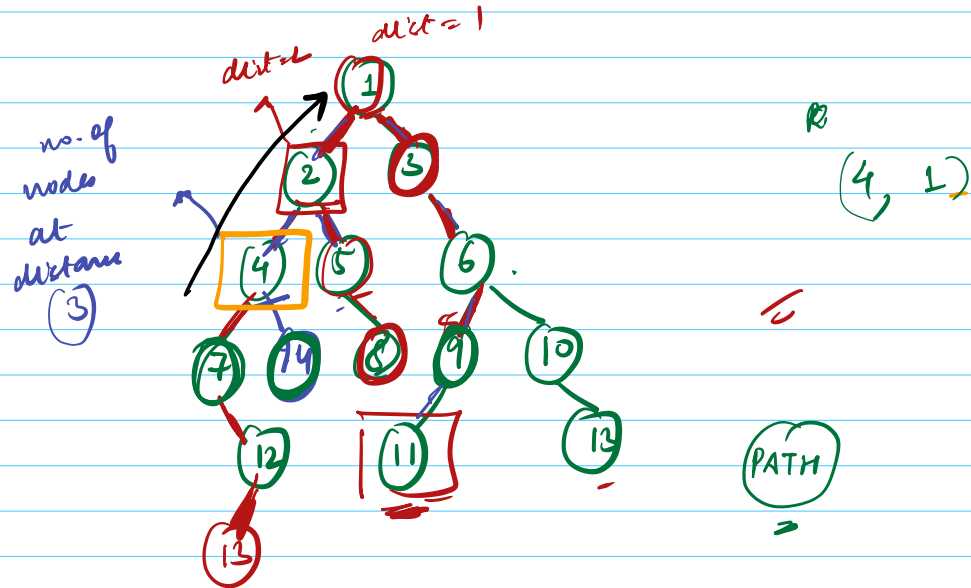
2

+

nodes at distance
2 from
Node 3

1

Find nodes at distance K from a given Node.



Left

$K=4$

path = [S, C, B, A]

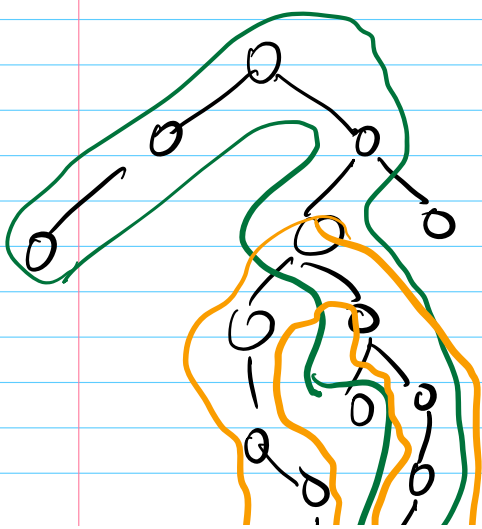
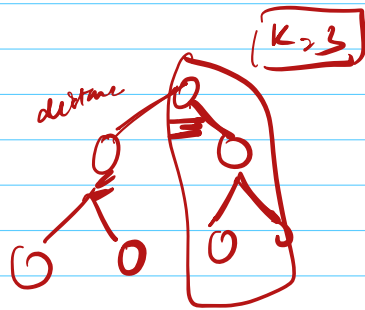
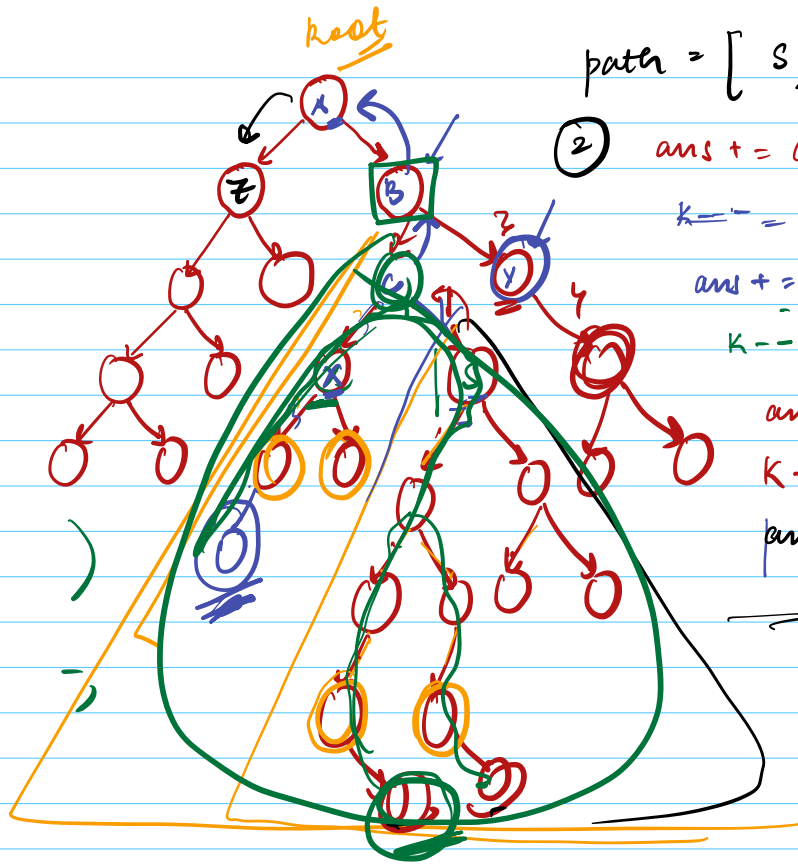
② $ans += count(S, 4)$

$k = 3$
 $ans += count(x, 2)$

$k = 2$
 $ans += count(y, 1)$

$k = 1$

$ans += count(z, 0)$



3



step - 1

Find ancestors of Node till Root
 ✓ (Get a list of all ancestor node references)

10 min

path = [S, C, B, A]

step - 2

ans += count (S, K)

step - 3

for (int i = 1; i < N; i++)

K--

moving to ancestor

if (K == 0) ans++ return.

if (path[i].left == path[i-1])

ans += count (path[i].right,

K-1)

else {

ans += count (path[i].left, K-1)

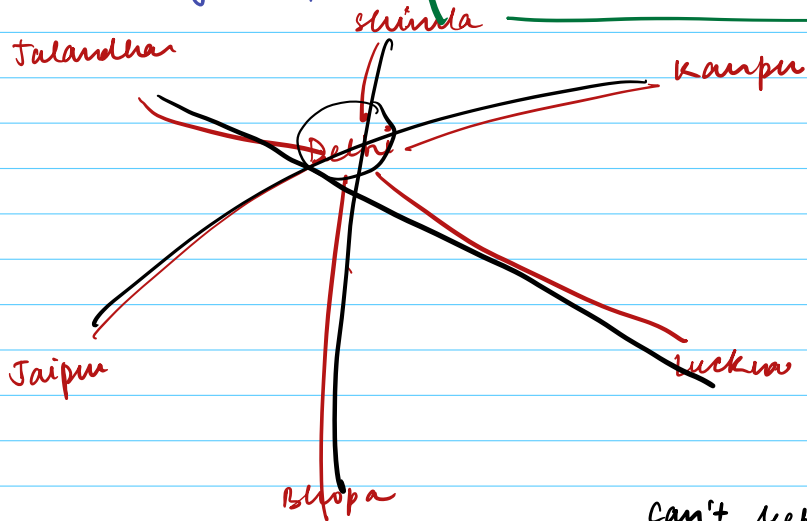
We are moving to child of ancestor

TC = O(N)
 SC = O(N)

Q

Find the longest path across the root.

$$H_L + H_R + 2$$



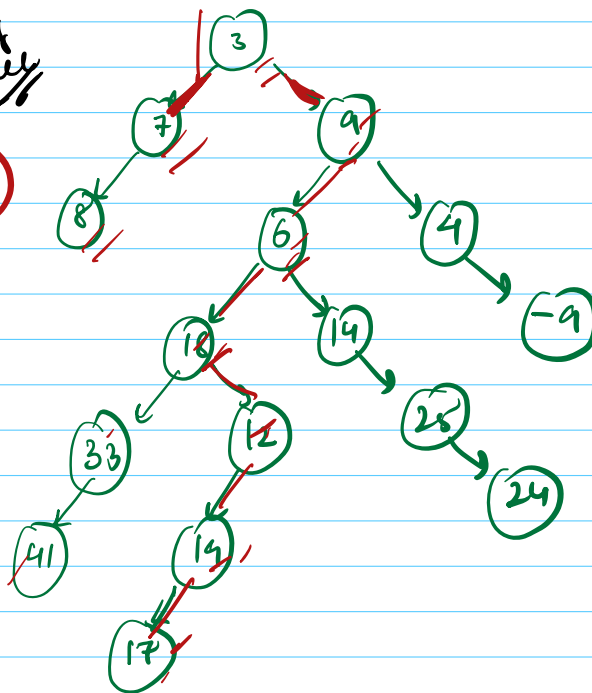
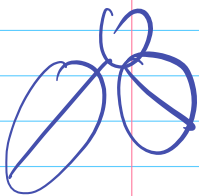
Diameter of binary tree

(2)

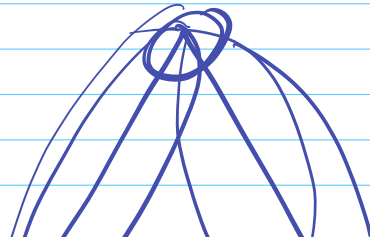
can't repeat any edge

$$(5+1)+2$$

$$H_L + H_R + 2$$



(6)





Q

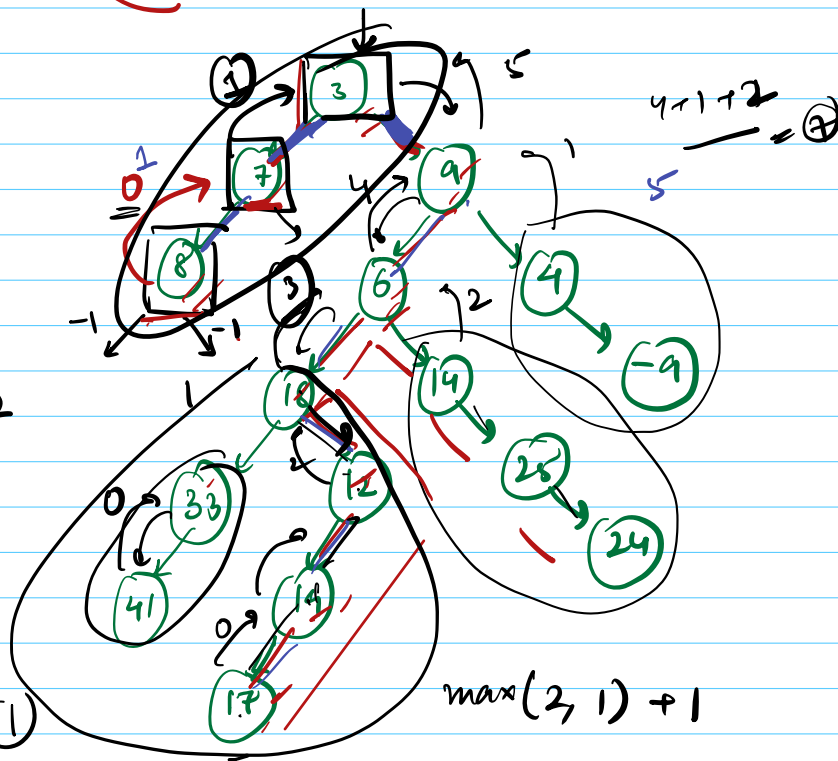
Longest path across any node /
Diameter of Tree.

$$\max (h_L + h_R + 2) \text{ } \forall \text{ node}$$

diameter = 0

$$0 + (-1) + 2 = 1$$

$$0 + (-1) + 2 = 1$$



$$(1 + 2) + 2 = 5$$

$$1 + 5 + 2 = 8$$

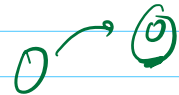
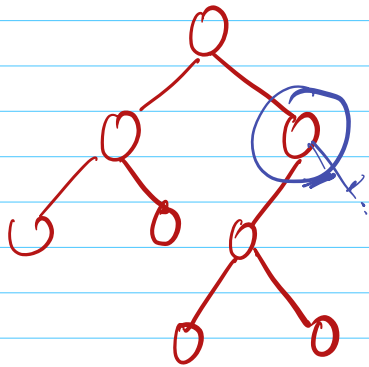
$$3 + 2 + 2 = 7$$

Why

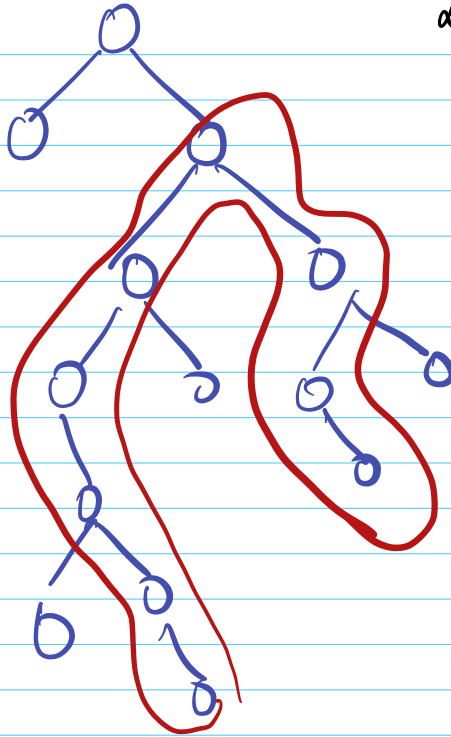
Diameter = 0

for a single node neighbor is 0

```
int height ( root )  
if ( root == null ) return -1  
HL = height ( root -> left )  
HR = height ( root -> right )  
( diameter = max ( diameter ,  
                    HL + HR + 2 ) )  
return max ( HL , HR ) + 1
```

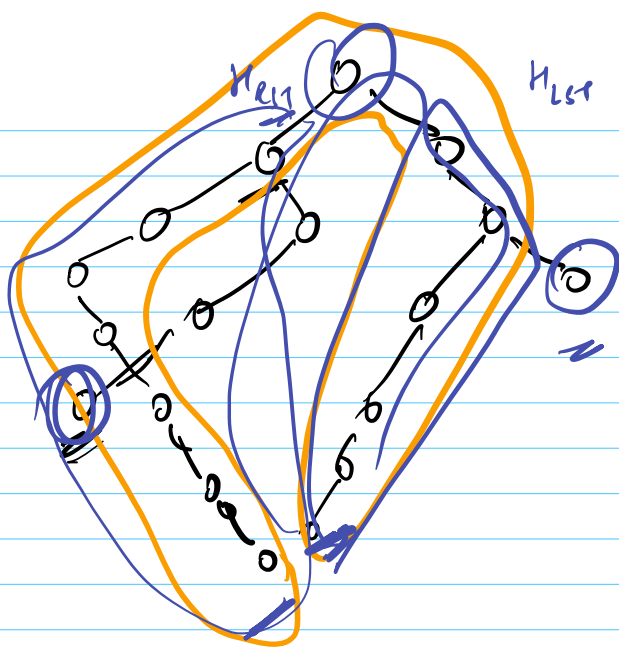


-1 to 1



01 name
 ↓
 start from
 any
 ↓
 go till
 any
 ↓
 count
 max
 distance
 without
 repetition
 in
 c++

max path sum $\rightarrow$ HW

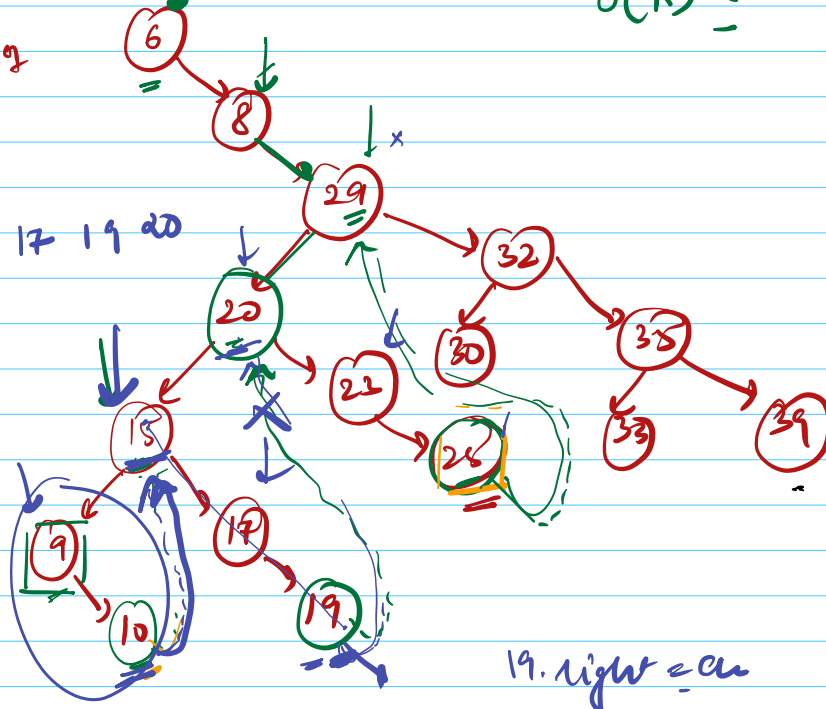


Morris Inorder Traversal

=> $O(N)$ TC
 $O(1)$ SC

LST Root RST

[6] 8 9 10 17 19 20



cur = root

while (cur != NULL)

```
if ( ! cur->left ) {
    print( cur->data )
    cur = cur->right
}
```

else {

pred = cur->left

while (pred->right) {
 pred = pred->right

}
 pred->right = cur

pred->right!
 = cur

```

    }
    {
        prev = right;
        cur = cur - left;
    }
    else {

```

```

    pred. right = NULL
    print ( cur )
    cur = cur. right

```